FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

IT0049
(WEB SYSTEM TECHNOLOGIES)

TECHNICAL FORMATIVE ASSESSMENT
1

FROM ZERO TO FOUR PAGES: YOUR FIRST CODEIGNITER APPLICATION

<table>
<tr><td>Student Name / Group Name:</td><td>Gil Roan L. Faminialagao</td></tr>
<tr><td>Members (if Group):</td><td><table><tr><th>Name</th><th>Role</th></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr></table></td></tr>
<tr><td>Section:</td><td>TW31</td></tr>
<tr><td>Professor:</td><td>Mr. John Benedict Enriquez</td></tr>
</table>

PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Analyze a complex problem and identify and define the computing requirements appropriate to its solution. [PO: B]

COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- Understand the advanced principles of web development using web frameworks and MVC architecture. [CLO: 1]

INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Install CodeIgniter using Composer and configure the application's base URL.
- Create routes for a multi-page website.
- Build controllers that respond to those routes.
- Create views and pass data to them.
- Use a static PHP array as a temporary data source for a listing page.

BACKGROUND INFORMATION

Before a web application can talk to a database, it needs a clear structure for turning a URL into a page. CodeIgniter organizes this using the Model-View-Controller (MVC) pattern: routes decide which controller method runs for a given URL, the controller decides what to do, and the view renders the HTML the browser receives. This activity focuses on the routing, controller, and view layers only — the data layer comes in the next module.

A static PHP array is a simple way to represent a list of records before a real database is introduced. It behaves like a temporary, in-memory table: you can loop over it in a view exactly the way you will later loop over database results, which is why this activity uses it as a stand-in.

GRADING SYSTEM / RUBRIC

CRITERIA	Description
Functionality & Requirements Completeness (40 points max)	All required pages and features are present and work as specified in the activity instructions.
Code Structure & Organization (25 points max)	Controllers, models, and views are separated correctly and follow the framework's conventions; code is readable and sensibly named.

CRITERIA	Description
Data Handling with Static Arrays (20 points max)	The static PHP array is structured sensibly, used consistently as the page's temporary data source, and looped through correctly in the view.
Documentation & Submission Quality (15 points max)	The GitHub repository is organized and complete, the README explains how to set up and run the project, and the hosted link is live and matches the submitted code.

LABORATORY ACTIVITY

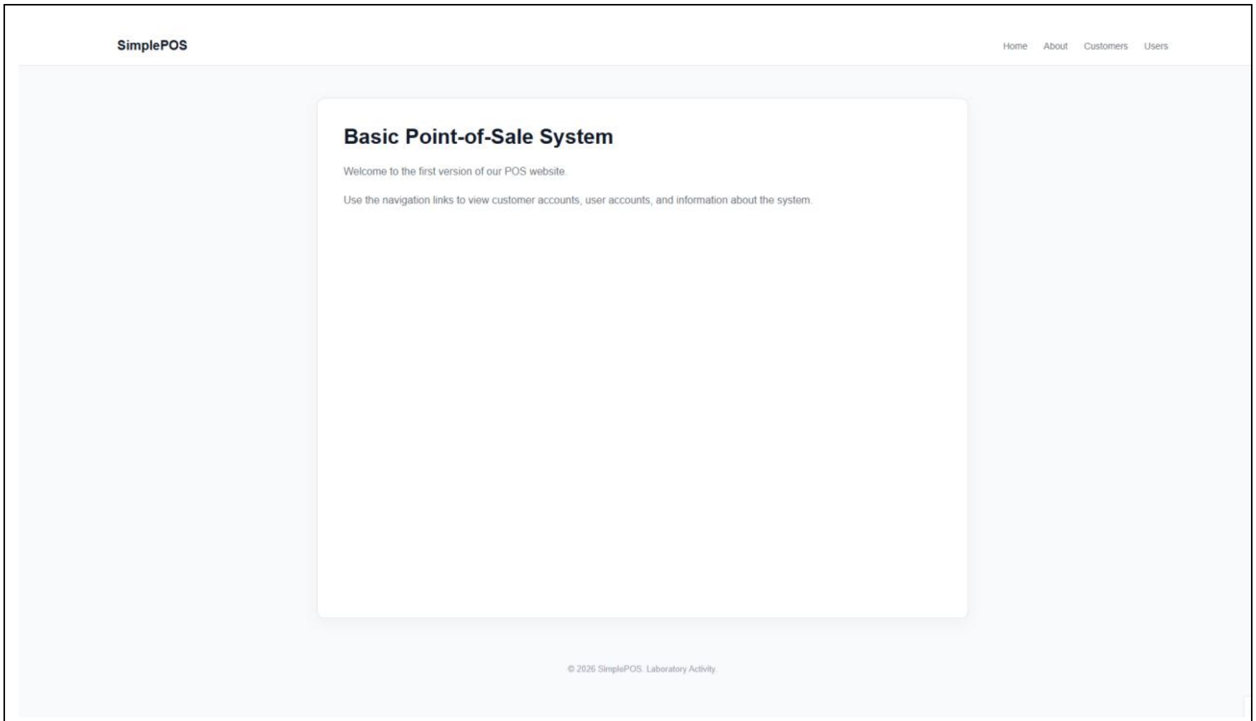
You will build the first version of a basic Point-of-Sale (POS) system: a four-page CodeIgniter website. The Customer Accounts and User Accounts pages will use a static PHP array as a temporary data source — no database is involved yet.

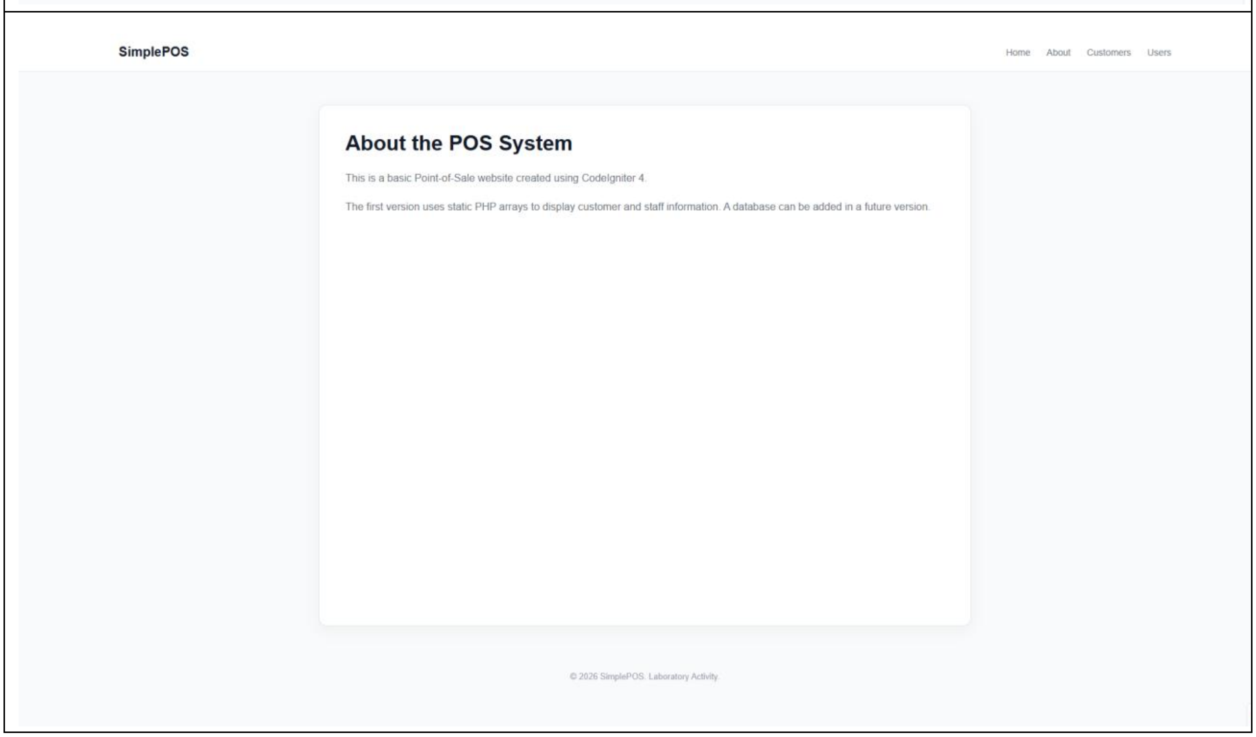
Required pages: a landing page (/), an about page (/about), a Customer Accounts page (/customers) listing customer records (full name, email, phone) from a static array, and a User Accounts page (/users) listing user/staff records (username, full name, role) from a static array.

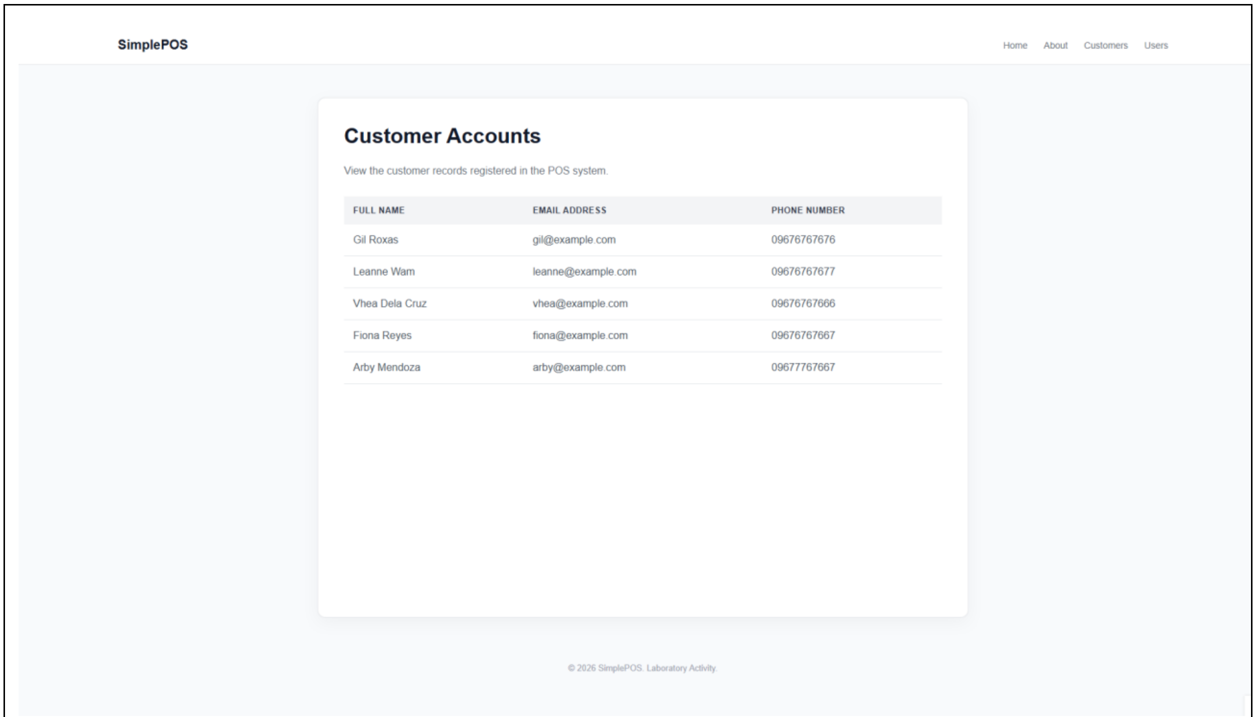
1. Install CodeIgniter 4 via Composer and confirm the base project runs in your browser.
2. Configure your app's base URL in .env and clean up the default welcome routes.
3. Create a Pages controller with methods for the landing and about pages, and register their routes.
4. Create a Customers controller and a Users controller. In each, define a static PHP array of at least 5 sample records as the method's temporary data source.
5. Build views for all four pages. In the Customer Accounts and User Accounts views, loop through the static array data with a foreach to display each record.
6. Add simple navigation links between all four pages and confirm every route works.

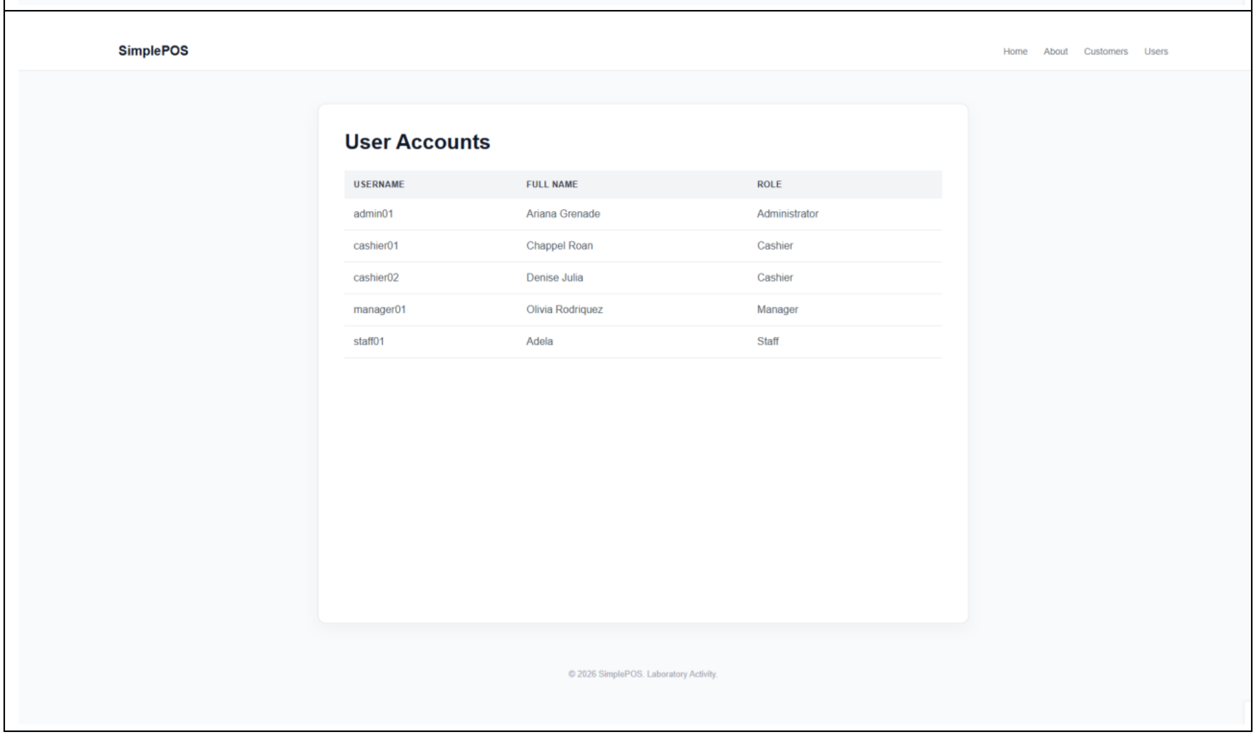
Submission Requirements:

- A link to a GitHub repository containing your raw project files, including your database export.
- A link to your hosted, working version of the application.


SimplePOS
Home
About
Customers
Users
Basic Point-of-Sale System
Welcome to the first version of our POS website.
Use the navigation links to view customer accounts, user accounts, and information about the system.
© 2026 SimplePOS. Laboratory Activity.


SimplePOS
Home
About
Customers
Users
About the POS System
This is a basic Point-of-Sale website created using CodeIgniter 4.
The first version uses static PHP arrays to display customer and staff information. A database can be added in a future version.
© 2026 SimplePOS. Laboratory Activity.





REFERENCES

- Manfield, A. (2021). Pro CodeIgniter: Learn how to create professional web-applications with PHP.
- CodeIgniter User Guide. Routing. https://codeigniter4.github.io/userguide/incoming/routing.html

- CodeIgniter User Guide. Controllers. https://codeigniter4.github.io/userguide/incoming/controllers.html